

File Transformation: Squaring Numbers

A step-by-step guide to reading, processing,
and writing numerical data in Python.

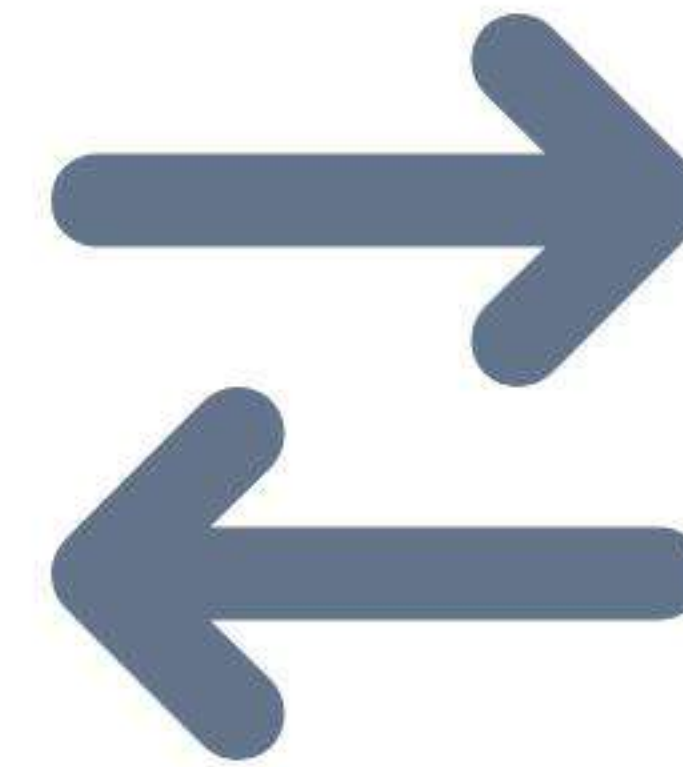


The Challenge

Goal

Read a file `numbers.txt` containing integers, calculate the square of each number, and write the results to `squared.txt`.

- ✓ **Input:** One integer per line.
- ✓ **Process:** Square each number (x^2).
- ✓ **Output:** One result per line.
- ✓ **Constraint:** No type errors allowed.



Visualizing the Data Flow



The Core Problem: Type Conversion

Strings vs. Integers

When you read from a file, Python gives you **strings** (text), even if they look like numbers.

```
"5" ** 2 → TypeError!
```

We must convert the string to an integer before doing math.



Solution 1: The Iterative Approach

Line-by-Line Processing

We assume the file might be very large, so we process it one line at a time.

1. ✓ Open both files (Input for reading, Output for writing).
2. ✓ Loop through each line of the input.
3. ✓ Convert, calculate, and write immediately.



Flowchart: Iterative Logic



(Repeats until End of File)

Code: Iterative Solution

```
def square_numbers_iterative():
    try:
        with open('numbers.txt', 'r') as infile, \
            open('squared.txt', 'w') as outfile:

            for line in infile:
                # Strip whitespace and convert
                num = int(line.strip())

                # Calculate square
                squared = num ** 2

                # Write back as string with newline
                outfile.write(f"{squared}\n")

    except ValueError:
        print("Error: Non-integer data found.")
```

Key Takeaways

- ✓ `line.strip()`: Removes the `\n` from the input line.
- ✓ `int()`: Converts text to number.
- ✓ `write(... \n)`: Adds the newline back for the output file.

Solution 2: The Batch Approach

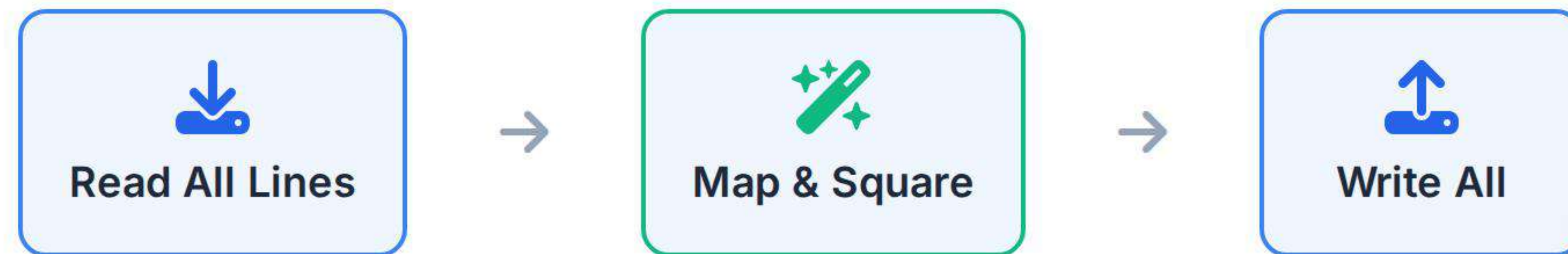
"Read All, Transform All"

If the file is small, we can load it all into memory. This allows us to use Python's concise List Comprehensions.

We transform the entire list of lines into a list of squared strings in one go.



Flowchart: Batch Logic



Code: Batch Solution

```
def square_numbers_batch():
    try:
        with open('numbers.txt', 'r') as f:
            lines = f.readlines()

        # Create list of formatted strings
        results = [
            f"{int(line)**2}\n"
            for line in lines
            if line.strip() # Skip empty lines
        ]

        with open('squared.txt', 'w') as f:
            f.writelines(results)

    except ValueError:
        print("Invalid data.")
```

Key Takeaways

- ✓ `readlines()`: Gets everything as a list.
- ✓ `[... for x in y]`: Compact loop.
- ✓ `writelines()`: Optimized for writing lists of strings.

Comparing Approaches

Feature	Iterative (Loop)	Batch (List Comp)
Memory Usage	Low (One line at a time)	High (Loads full file)
Speed (Large Files)	Better (Streaming)	Worse (Memory bottleneck)
Code Style	Explicit & Simple	Pythonic & Concise
Best Use Case	Processing Data Logs	Small Config Files

Handling Errors

Constraint: "No Type Errors"

What if the file contains "hello" or an empty line?

```
try:  
    num = int(line)  
except ValueError:  
    pass # Skip bad data
```

Wrapping the conversion logic in a try-except block ensures the program doesn't crash on bad input.



Summary

- ✓ **Read:** Use `int()` to convert string inputs to numbers.
- ✓ **Process:** Perform math (`** 2`).
- ✓ **Write:** Convert back to string and append `\n`.
- ✓ **Safety:** Always use try-except when converting user data.

